UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

COS730 – Assignment 2

From Behavioural Models to Optimised Implementation

Ruan Esterhuizen (u23532387)

May 2026

Introduction

This assignment involves implementing an intelligent submission and review system from a given design, which is functionally correct, but suboptimal. This design is then critically evaluated and optimized. The optimized system is implemented. Finally, the original and optimized systems are compared according to various metrics to assess the effectiveness of the optimizations.

Assumptions

The following assumptions have been made during the initial implementation of the system and holds throughout this assignment.

1. Activation bars in the sequence diagram stretch from the first function call to the last. That is, a class is instantiated directly before the first function call to that class is made.
2. To simulate reviewers scoring papers, a random score is generated when the *assignReview()* function is called.
3. A notification service has not been implemented, this behaviour has been mocked by printing the notification content to the console and a *sleep()* function mocks a call to some external service, such as SendGrid.
4. A real-world system would likely receive research outputs as PDF documents, but for the purposes of this assignment, JSON documents were used to represent research outputs. An example document can be viewed in Appendix 2.
5. The "Database" in the sequence diagram is assumed to be a class that handles database connection and interactions, not the database directly.

Source Code Access

The source code for this assignment can be accessed here: github.com

The original and improved implementations of the system can be found in the "Original" and "Improved" subfolders respectively.

The ReadME file contains all necessary instructions to run the system.

Task 1: Baseline Implementation

The given system design has been implemented with complete adherence to the sequence diagram.

Python was used to implement the system. It was chosen for its readability and abundance of built-in libraries. SQLite was used to implement a simple relational database. It was chosen for its simple configuration and seamless integration with Python programs (via the sqlite3 module). Although the system has been simplified where possible, implementing database interactions was important for analysing how the system handles storage operations. A simple GUI for researchers to submit artefacts is implemented using Tkinter.

1.1. Adherence to diagram elements

To illustrate traceability between diagram elements and code, each method that appears in the sequence diagram include a print statement such that messages are printed in the format: <recipient>:<message>. For example:

```
class Validator:
    def validateFormat(self, data) -> bool:
        print("V: Validating format")
```

This way, console output shows that the implemented system adheres to the sequence diagram.

```
UI: Submitting Data to SubmissionController
SC: Submitting research output
V: Validating format
DB: Saving submission
RM: Getting available reviewers
DB: Fetching reviewers
RM: Reviewing conflicts
RM: Checking Workload
R: Assigning Review (Prof. R Starr - Automating the REPA protocol)
R: Assigning Review (Dr. H Williams - Automating the REPA protocol)
EM: Starting Evaluation
EM: Submitting score
DB: Saving score
EM: Submitting score
DB: Saving score
EM: Calculating average
EM: Checking consensus
EM: Applying Rules
NS: Notifying Acceptance (Automating the REPA protocol)
```

Names of system components have been abbreviated for readability. See the attached glossary for explanations of abbreviations.

1.2 Design Assumptions

Some behaviour is not included in the system design; thus, the following assumptions have been adopted.

Validating format: A research output (formatted as a JSON document) has valid format if the following conditions are met:

- JSON must be valid, e.g. a document with an extra '}' at the end is invalid
- All keys must be present with non-empty values. The required keys are as follows: *["title", "author", "date", "group", "supervisor", "abstract", "keyword"]*
- The "group" attribute (representing a research group) must contain one of the following values: *["CIRG", "SSFM", "CSEDAR", "DSfSI", "NICOG", "DigiForS"]*

Filtering Reviewers: For a reviewer to be considered available to review a given artefact, the following conditions must be met:

- No conflicts: All reviewers must be affiliated with the same research group as the artefact.
- Checking workload: After filtering out conflicts, one reviewer is removed from the list, unless there is only one reviewer in the list. This process has been simplified for the purposes of this assignment.

Determining evaluation outcome: Reviewer scores (out of 10) are randomly generated for each reviewer.

- Approved: Average score must be greater than or equal to 7.5
- Rejected: Average score less than 7.5
- Revision: The difference between the minimum and maximum score is greater than 5. This overwrites the rules for approval or rejection.

Task 2: Design Analysis

The implemented system displays various issues, mainly relating to violations of responsibility assignment principles and low quality of behaviour modelling.

2.1 Redundant message calls

1. **submitScore() and saveScore()**

   The EvaluationManager collects the all the reviewers' scores and saves them to the database as soon as the score is received. This means that for N reviewers, N calls are made to the database. This is redundant and can be replaced with a single call to the database with all the scores when all of them are collected.

2. **getAvailableReviewers() and fetchReviewers()**

   According to the GRASP Information Expert pattern, a message call to an intermediary object (ReviewerManager) is redundant if it does not add any logic. This is the case because the SubmissionController calls ReviewerManager, which calls Database, the ReviewerManager adds no logic; thus, the fetchReviewers() message call is redundant. Either this call should be eliminated and SubmissionController should call Database directly, or the SubmissionController shouldn't have a list of reviewers at all. The latter is a better solution, since it adheres to responsibility assignment principles.

2.2. Violations of good responsibility assignment

The **SubmissionController** is an example of a bloated controller that does everything itself. It should only be responsible for submitting an artefact, but in the original implementation it is responsible for validating format, submission, assigning reviewers and starts the evaluation. This is a violation of the Single Responsibility Principle. This introduces high coupling into the system. Properly using the GRASP Controller pattern should alleviate this.

2.3. High coupling / low cohesion areas

Areas of high coupling and low cohesion greatly harm the quality of a system, since it makes the system difficult to test and maintain. High coupling means that modules are closely connected and changes in one module may affect other modules. A class with low cohesion does not fulfil a single well-defined purpose.

The area of the system that demonstrates the highest degree of coupling is the **SubmissionController**. This can be seen in the sequence diagram: its lifeline touches the lifeline of 5 other classes (Validator, Database, ReviewerManager, Reviewer, EvaluationManager). This means that if any of these classes would change, the SubmissionController should have to change as well, these components should also be mocked to efficiently unit test the SubmissionController. This tight coupling between system components is mainly the result of poor responsibility assignment principles.

The **EvaluationManager** is an area of low cohesion in the system. It is responsible for collecting scores from reviewers, determining evaluation outcome and sending a notification. This violates the single responsibility principle and leads to low cohesion, since the class fulfils a collection of unrelated tasks that should be delegated to other components. For example, ReviewerManager should be responsible for collecting scores and the controller should send notifications.

2.4. Inefficient control flow

The database contains a list of reviewers, which is then filtered. There is a case where the *filteredReviewers* list is empty, e.g. the database does not contain any reviewers affiliated to the research group that an artefact is tied to. In this case, the researcher is not notified of this being the case and the system proceeds to start an evaluation, which is highly redundant. This is a case of poor quality of behavioural modelling, which introduces an inefficient control flow to the system. The notification received by the researcher will also be ambiguous in this case, it is unknown whether the “revision” notification is due to low scores by reviewers or no reviewers being present at all.

In a real-world system, reviewers submitting scores is a manual process, but it has been modelled synchronously. The entire system stalls while reviewers are evaluating the artefact, which is an example of low quality of behavioural modelling.

2.5. Poor handling of decision logic

The goal of a sequence diagram is to model interactions between objects, not expose the internal logic of classes. This is the case in **EvaluationManager** and **ReviewManager** with the *filterConflicts()* & *checkWorkload()* and *calculateAverage*(), checkConsensus() & *applyRules()* self-calls. This is considered poor quality modelling of behaviour.

The design does not explicitly present that the **EvaluationManager**’s *applyRules()* method drives the outcome of the evaluation, which is used in the subsequent alt fragment. This is low quality behavioural modelling. Moreover, it is also an improper implementation of the GRASP Controller pattern, since the controller should receive the outcome of the evaluation, not the NotificationService; thus, it is also a violation of responsibility assignment principles.

Another example of poor behavioural modelling, leading to poor handling of decision logic, is the fact that *validateFormat()* only returns if the format is valid or not. It is not communicated to the researcher why the format is invalid if it is. This is a problem because multiple rules are used to evaluate whether the format is valid or not.

Task 3: Decision Modelling

3.1 Rules and Actions

Based on the beforementioned critiques of the decision modelling in the system design and assumed rules that were specified in the system implementation, the following conditions and actions can be deduced:

Conditions:

For an artefact to be accepted, the following conditions must hold

C1: Output format must be valid

C2: More than 1 reviewer is available

C3: Difference between minimum and maximum score less than 5

C4: Average score greater than 7.5

Actions:

A1: Accept artefact

A2: Reject (Invalid format)

A3: Reject (No reviewers available)

A4: Revision (Consensus was not reached)

A5: Reject (Average score too low)

3.2 Decision Table

A simplified decision table can be created based on the rules and actions.

Conditions / Rules	R1	R2	R3	R4	R5
C1: Format	N	Y	Y	Y	Y
C2: Reviewers	-	N	Y	Y	Y
C3: Consensus	-	-	N	Y	Y
C4: Average score	-	-	-	N	Y
---	---	---	---	---	---
Action	A2	A3	A4	A5	A1

3.3 Discussion

C2 and A3 have been added to the decision table to eliminate ambiguity in the initial system design and implementation. In the case where there are no reviewers available, the system will

no longer return an ambiguous revision notification, but the researcher will explicitly be notified that there are no reviewers available.

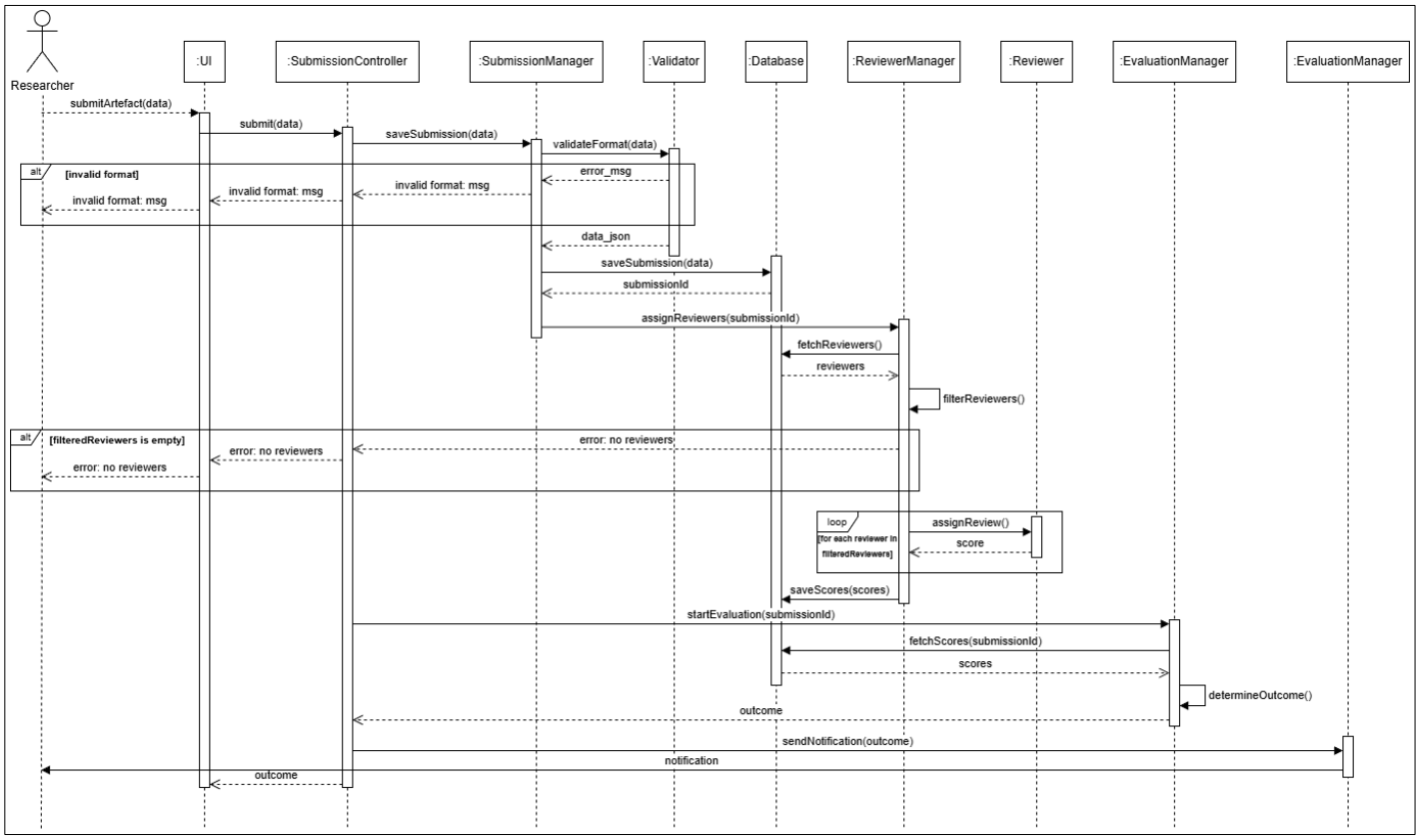
Excluding C2 and A3, all other rules map to the existing system design: C1 and A2 correspond to the *validateFormat()* method and *"return error"* segment of the sequence diagram. C3, C4, A1, A4 and A5 all map to the EvaluationManager's logic behind determining the outcome of the review, i.e. the final "alt" fragment in the sequence diagram.

The decision table also eliminates the clarity of which conditions must be met for an artefact to be accepted, and which conditions cause rejection or evaluation.

Task 4: Sequence Diagram Optimization

The following sequence diagram was created after applying the critiques of the initial system design in Task 2 and the enhanced decision modelling in Task 3.

A high resolution image of the diagram below can be viewed on the GitHub Repository.



4.1 Key design changes

- Adherence to a layered architecture to reduce coupling between components.
- Better decision modelling for errors.
- Simplified the bloated SubmissionController by abstracting some of its functionality to a new SubmissionManager class.
- Improved database interactions and applied the singleton pattern.

These changes are explained in more detail and justified in Task 5.

Task 5: Optimised Implementation

5.1 Adherence to sequence diagram

Using the same method from Task 1, it can be shown that the implemented system adheres to the sequence diagram in Task 4.

```
UI: Submitting data to SubmissionController
SC: Submitting research output
SM: Saving Submission
V: Validating format
DB: Saving submission
RM: Assigning Reviewers
DB: Fetching reviewers
RM: Reviewing conflicts
RM: Checking Workload
R: Assigning Review (Prof. Ringo Starr)
R: Assigning Review (Dr. Hayley Williams)
DB: Saving scores
EM: Starting Evaluation
DB: Fetching Scores
EM: Determining Outcome
NS: Notifying Accepted (Automating the REPA protocol)
```

5.2. Functional equivalence with original system

Despite adding new components and changing behaviour, both the original and improved systems are functionally equivalent. If a valid research artefact is submitted and there are reviewers available, both systems will save that submission, along with its scores to the database and notify the researcher of the outcome of the evaluation.

5.3 Responsibility assignment

The original system exhibited tight coupling and low cohesion between system components, mainly due to violations of responsibility assignment principles. To avoid this in the improved

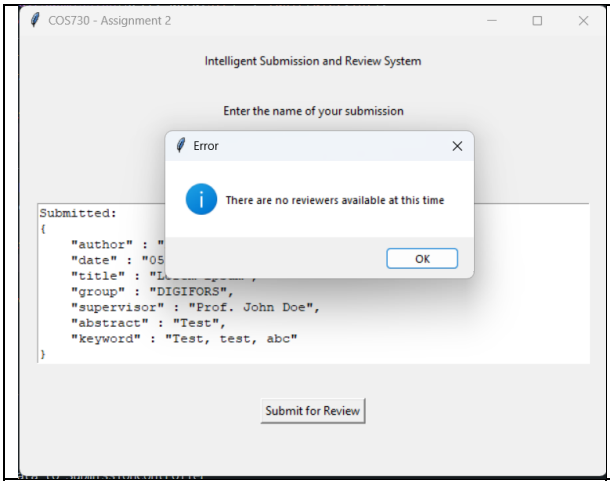
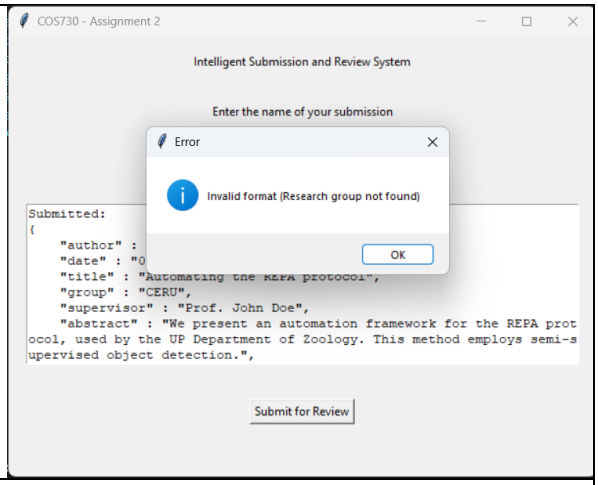
implementation, the responsibilities of each component were defined in the table below and strictly adhered to during implementation.

Component	Layer	Responsibility
UI	Presentation	Receiving input and presenting output to the user.
SubmissionController	Controller	Orchestrates the flow of the program, without implementing any business logic.
SubmissionManager	Expert	Acts as the expert on Submissions. It applies the business logic of validating artefacts and sending it to the persistence layer.
ReviewerManager	Expert	The only component that directly interacts with the Reviewer. The logic for filtering reviewers and assigning reviewers is handled by this component.
EvaluationManager	Expert	Collects scores from the persistence layer and determines the outcome of the evaluation, based on predefined rules.
Database	Persistence	The only component that directly interacts with the database itself. It receives data from the expert layer and stores it, without applying any logic to the data that it persists.
Reviewer	Domain	Represents a single reviewer and can only acted upon by the information expert on reviewers, which is the ReviewerManager.
Validator	Utility	Abstracts validation logic from the SubmissionManager. This is done for separation of concerns, the SubmissionManager does not have to implement validation logic.
NotificationService	Utility	Abstracts sending notifications. This is especially important, since this type of task usually depends on some external service (such as SendGrid).

Adhering to this layered approach means that messages flow in a predefined manner: The controller initiates a process, the manager applies some business logic, and possibly retrives/stores data using the persistence layer. This was not the case in the original system, where the SubmissionController directly communicated with the persistence layer, for example.

5.4 Enhanced behavioural modelling

As mentioned in previous tasks, the original system had various issues regarding the quality of behavioural modelling. For example, it did not account for a case where there are no reviewers available, the system would complete a series of redundant steps and return an ambiguous message at the end. The design also did not model any communication of errors to the UI. This has been fixed in the improved system: in the event of an error, the system communicates the error to the user, via the UI and terminates. The following screenshots show errors being shown to the user, this behaviour is completely absent in the original implementation.

	
Corresponds with A3 in the decision table	Corresponds with A2 in the decision table

5.5 Database interactions

Creating an instance of the Database class involves some amount of overhead, such as creating a connection to the database itself and creating/updating tables if required. In the original implementation, this process was repeated each time an instance of the Database class is created, which is redundant.

To minimize this overhead, the singleton pattern is applied to minimize this overhead. A single instance is created, which is used throughout the runtime of the program. This pattern has been implemented using the *__new__(...)* method.

Task 6: Empirical Evaluation and Comparison

The goal of this task is to compare the original implementation of the system (Task 1) with the improved implementation (Task 5). The two systems have been compared using various tools and metrics, as discussed in the following sections.

6.1 Number of interactions

The number of interactions for each system was considered because it can give an indication of how the system is structured.

The most straightforward way to do this is counting the number of method calls in the sequence diagram. For N reviewers, the original system executes **16+(N*2)** method calls and the improved system executes **13+N** calls.

The number of method calls have also been counted at runtime using a simple Python script, which has been included in Appendix 4. The script makes use of the *pstats* library to count the number of interactions.

The table below shows the interactions count for the original and improved systems. The manual count from the sequence diagram shows a case of N reviewers and the runtime count is for 2 reviewers.

	Original	Improved
Manual count	16 + (N*2)	13 + N
pstats	43	45

Note that pstats includes methods that are not included in the sequence diagram, such as constructors and helper methods that were implemented, but not illustrated in the diagram.

Counting interaction at runtime proved that the improved system has about the same number of interactions as the original. This is possibly due to the newly added component in the system. For example: the original SubmissionController directly saved the submission to the database, whereas the improved SubmissionController delegates saving to the database to the SubmissionManager. Although this approach has more interactions, it demonstrates better adherence to responsibility assignment principles.

It is also worth noting that not all method calls have an equal impact on the performance of the system, for example: the original system saved individual scores to the database, where the improved system first collects all scores and saves them all at once. These approaches will have approximately the same number of method calls, but the latter would be more efficient, since it contains less interactions with the database, which is a relatively expensive operation.

6.2. Execution time

The time it takes for a system to complete a task is one of the most important metrics to consider when evaluating the quality of a system.

All runtimes presented below are the average over 15 runs, this was done to minimize the impact of random noise such as CPU scheduling, background tasks, disk caching, etc.

Case 1: Positive Case

A valid research artefact shown in Appendix 2 was used for this case. The database was seeded with 6 reviewers, where 2 review the submission.

	Original	Improved
Average runtime (ms)	64.786	44.294

The improved implementation delivered a **31.63%** improvement in runtime for this case. This can mainly be attributed to the reduction of unnecessary interactions and application of the singleton pattern for the Database component.

Case 2: No reviewers

A valid research output was also used for this case, but the “research_group” value was set such that there are no reviewers available. This case was considered to evaluate the effectiveness of adding the enhanced decision modelling, as explained in previous Tasks.

	Original	Improved
Average runtime (ms)	17.617	6.385

For this case the improved implementation delivers a **63.76%** improvement from the original implementation. Mainly due to the enhanced decision modelling in the improved system. The

improved system terminates with an error message, whereas the original system performs redundant steps and returns an ambiguous response.

6.3. Code complexity

Various static code analysis tools exist to evaluate the complexity of a system. Radon is a popular tool used for Python systems.

Radon was used to evaluate the cyclomatic complexity of each system. Cyclomatic complexity measures the number of unique paths through the code. Lower complexity means that the codebase is easier to modify and maintain.

The following table compares the average complexity score for each system, calculated using the "**radon cc -a -s**" command.

	Original	Improved
Average cyclomatic complexity score	2.43	2.62

Although the improved system has a complexity score that falls in the A rank (lowest risk), the original system presents a slightly better score. This may be due to the implementation of decision modelling and error handling behaviour that was implemented in the improved system but is absent from the original implementation.

For completeness, the full output from the improved system has been included in Appendix 3.1.

6.4. Maintainability

Quantitative Analysis

Radon also provides a tool for statically analysing the maintainability of a codebase. It considers every source code file and computes a Maintainability Index score, where 100 is the highest possible value. The goal is to maximize maintainability to make it as easy as possible to modify or improve the system over time.

The following outputs were generated after using the "**randon mi**" command on each system.

Original	Improved
…/Database.py - A (61.14) …/EvaluationManager.py - A (47.59) …/main.py - A (100.00) …/NotificationService.py - A (100.00) …/Reviewer.py - A (100.00) …/ReviewerManager.py - A (78.54) …/SubmissionController.py - A (66.73) …/UI.py - A (56.47) …/Validator.py - A (77.74)	…/Database.py - A (59.45) …/EvaluationManager.py - A (53.72) …/main.py - A (100.00) …/NotificationService.py - A (100.00) …/Reviewer.py - A (100.00) …/ReviewerManager.py - A (79.40) …/SubmissionController.py - A (100.00) …/SubmissionManager.py - A (89.47) …/UI.py - A (53.56) …/Validator.py - A (77.74)
Average: 76.96	Average: 81.47

The improved system has a higher average maintainability score than the original, mainly due to the reduced complexity of the SubmissionController. This is mainly because the original SubmissionController implemented business logic and was tightly coupled with other system components, this was elevated by abstracting the business logic to the newly created SubmissionManager in the improved implementation.

Qualitative Analysis

Code analysis tools provide a good indication of the maintainability of a system by evaluating the structure of the source code. However, quantitative analysis does not fully capture how easy it is for a developer to fix bugs and add new features. Thus, a qualitative analysis of each system is required.

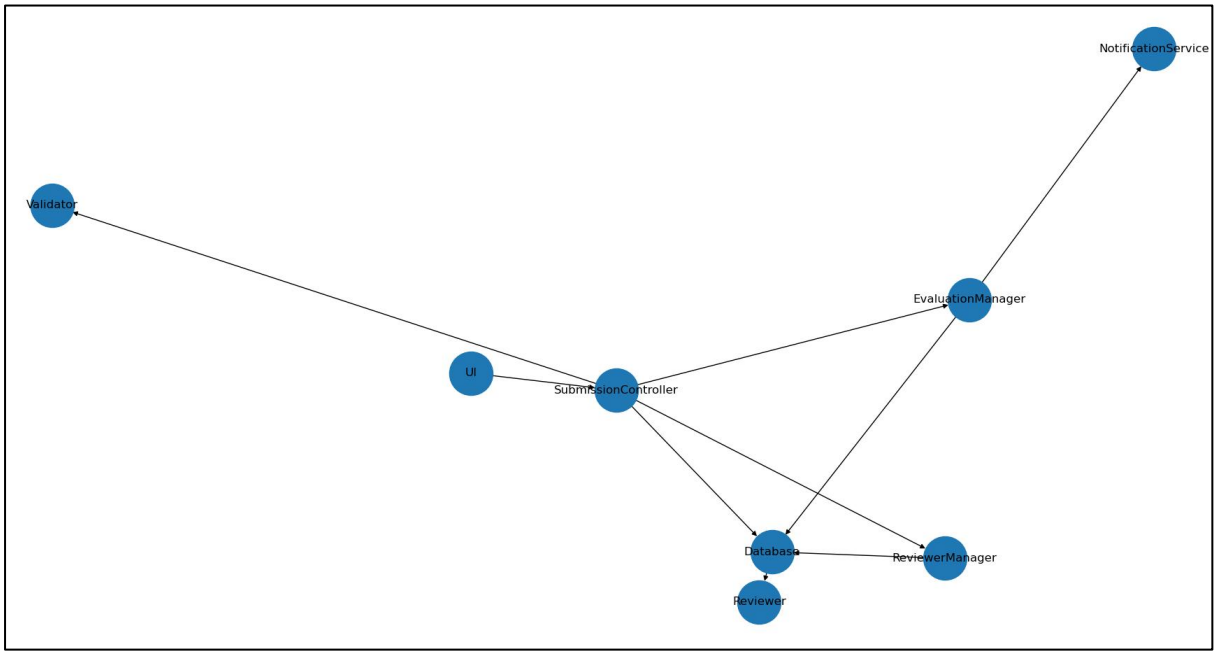
The improved system demonstrates better maintainability, due to its separation of concerns. A layered approach was introduced, which means that each class has a strict purpose and domain that it should operate in (as shown in Task 5). For example: if a developer who is unfamiliar with the system had to add new business logic to how submissions are handled. In the original system, they would open the SubmissionController and be met with a bloated controller that handles logic beyond the scope of submissions. In the improved system, the SubmissionManager exclusively handles all business logic related to submissions.

Qualitative measures of maintainability usually also include factors such as quality of documentation, which is beyond the scope of this assignment.

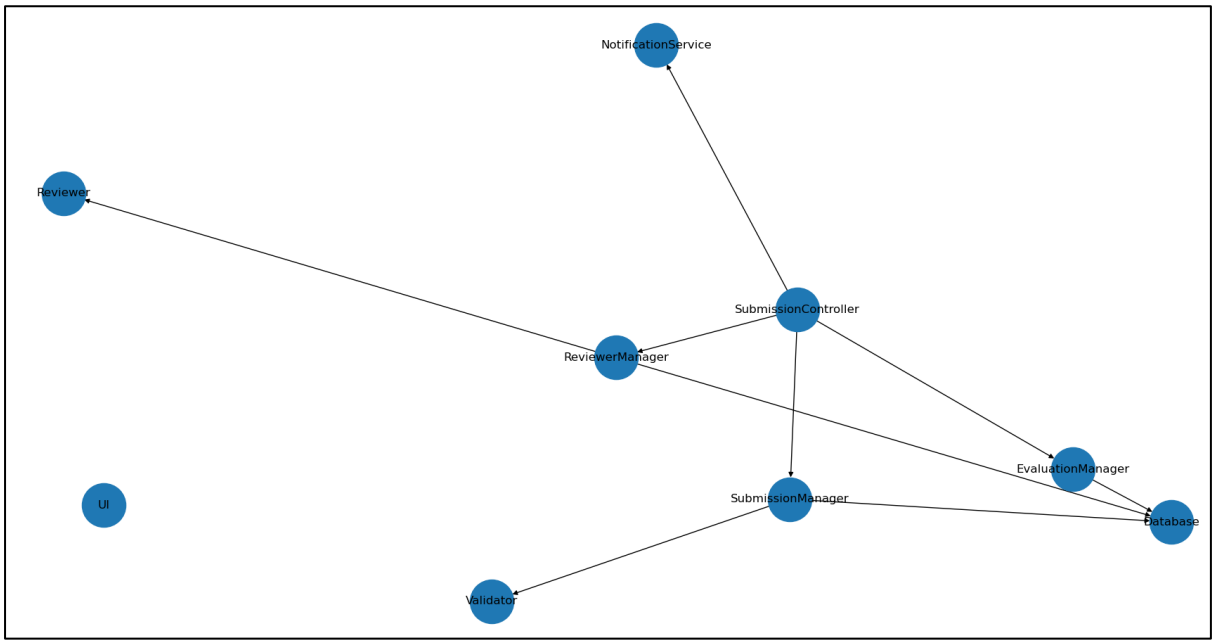
6.5 Dependency Graphs

To compare the cohesion and coupling in the systems, a Pyhton program was used to analyse the source code for each system and draw a dependency graph. An edge from component X to component Y means that component X references Y in any way, such as calling one of its functions or instantiating it, i.e. if there is an arrow from one class to another, the former class uses the latter.

Original



Improved



This exemplifies that the improved system adheres to responsibility assignment principles: domain objects (Reviewer) are only used by their information experts (ReviewerManager). The controller only interacts with managers and only managers interact with the database.

It can however be noted that the SubmissionController is still coupled with all of the managers.

6.6 Conclusion

The optimizations that were implemented were aimed at reducing unnecessary interactions between system components, better responsibility assignment and improving decision modelling. These improvements were largely successful, since the system presents lower runtimes, especially for previously unhandled error cases. The codebase is more maintainable overall.

Trade-offs

In improving decision modelling, which enhanced error handling and communication of errors to the user, the overall complexity of the system was sacrificed. This can be seen in the analysis of cyclomatic complexity, where the original system slightly outperformed the improved one.

Possible improvements

The largest improvement that this system can benefit from is simplifying the SubmissionController even further. The dependency graph still shows that the improved SubmissionController is coupled with all components in the expert layer (SubmissionManager, ReviewerManager, EvaluationManager). This problem can possibly be elevated by introducing an intermediary layer that orchestrates passing data between the expert layers. That way, the controller only must initiate steps, not also pass data to the expert layer.

References

- KUNG. (2021). *Object-Oriented Software Engineering*
- MARSALL, PIETERSE. (2025). *Tackling Design Patterns*. cs.up.ac.za
- GeeksForGeeks. (2024, March 13). *GRASP Design Principles in OOAD*. geeksforgeeks.org
- GeeksForGeeks. (2026, January 17). *Singleton Pattern in Python.* geeksforgeeks.org
- GeeksForGeeks. (2025, September 27). *Cyclomatic Complexity.* geeksforgeeks.org
- Radon Documentation. radon.readthedocs.io

Appendix 1: System Component Abbreviations

Acronym	System Component
UI	UI (Graphical User Interface)
SC	SubmissionController
V	Validator
DB	Database
RM	ReviewerManager
R	Reviewer
EM	EvaluationManager
NS	NotificationService
SM*	SubmissionManager

* Component is only present in the Improved implementation

Appendix 2: Example Research Output

```
{
    "author" : "Ruan Esterhuizen",
    "date" : "05/05/2026",
    "title" : "Automating the REPA protocol",
    "group" : "CIRG",
    "supervisor" : "Prof. John Doe",
    "abstract" : "We present an automation framework for the REPA protocol...",
    "keyword" : "Elephant Conservation, Computer Vision, Pseudo Labels"
}
```

Appendix 3: Console Output

3.1 Code complexity analysis

```
ruane@apollo.../Assignment2$ radon cc Improved/ -s -a
Improved/Database.py
    M 93:4 Database.fetchReviewers - A (5)
    C 4:0 Database - A (4)
    M 144:4 Database.fetchScores - A (4)
    M 46:4 Database.saveSubmission - A (3)
    M 114:4 Database.saveScores - A (3)
Improved/EvaluationManager.py
    C 3:0 EvaluationManager - A (3)
    M 22:4 EvaluationManager.determineOutcome - A (3)
    M 4:4 EvaluationManager.fetchScores - A (2)
    M 11:4 EvaluationManager.checkConsensus - A (2)
    M 16:4 EvaluationManager.checkAcceptance - A (2)
    M 30:4 EvaluationManager.startEvaluation - A (1)
Improved/NotificationService.py
    C 3:0 NotificationService - A (2)
    M 4:4 NotificationService.sendNotification - A (1)
Improved/Reviewer.py
    C 3:0 Reviewer - A (2)
    M 8:4 Reviewer.assignReview - A (1)
Improved/ReviewerManager.py
    C 4:0 ReviewerManager - A (3)
    M 5:4 ReviewerManager.filterConflicts - A (3)
    M 33:4 ReviewerManager.assignReviewers - A (3)
    M 16:4 ReviewerManager.checkWorkload - A (2)
    M 21:4 ReviewerManager.fetchReviewers - A (2)
Improved/SubmissionController.py
    C 6:0 SubmissionController - A (2)
    M 7:4 SubmissionController.submit - A (1)
Improved/SubmissionManager.py
    C 4:0 SubmissionManager - A (4)
    M 5:4 SubmissionManager.saveSubmission - A (3)
Improved/UI.py
    C 5:0 UI - A (2)
    M 31:4 UI.submitData - A (2)
Improved/Validator.py
    C 3:0 Validator - B (9)
    M 4:4 Validator.validateFormat - B (8)

34 blocks (classes, functions, methods) analyzed.
Average complexity: A (2.6176470588235294)
```

Some entries such as constructors have been omitted.

Appendix 4: Interactions count script

```
import cProfile
import pstats
import io

def run():
    from main import main
    main()

pr = cProfile.Profile()
pr.enable()
run()
pr.disable()

stream = io.StringIO()
stats = pstats.Stats(pr, stream=stream)

stats.sort_stats("calls")
stats.print_stats("Improved")   # excludes library methods

print(stream.getvalue())
```